分享丨模运算的世界：当加减乘除遇上取模（模运算恒等式/费马小定理/组合数）

灵茶山艾府

65773 2024.05.14 2026.03.09 发布于 浙江 同步题目状态 重置当前页面题目状态

数学 数论 C C++ Java Python Python3 周赛 数据结构与算法 双周赛 力扣竞赛 学习分享

前言

某些题目，由于要计算的答案非常大（超出 64 位整数的范围），会要求把答案对 10^9+7 取模。如果没有处理得当的话，会 WA（错误）或者 TLE（超时）。

例如计算一堆数字的乘积，如果没有及时取模，乘法会溢出（例如计算结果超出 C++ 中 `long long` 的最大值），从而得到和预期不符的答案。对于 Python 来说，虽然没有溢出的问题，但大整数（big integer）之间的运算并不是 $\mathcal{O}(1)$ 的，可能会导致 TLE。

如何正确地取模呢？

729 110 1.6K

如果让你计算 1234×6789 的**个位数**，你会如何计算？

由于只有个位数会影响到乘积的个位数，那么 $4 \times 9 = 36$ 的个位数 6 就是答案。

对于 $1234 + 6789$ 的个位数，同理，$4 + 9 = 13$ 的个位数 3 就是答案。

你能把这个结论抽象成数学等式吗？

一般涉及到取模的题目，会用到如下两个恒等式，其中 $\bmod$ 表示取模运算（modulo），即编程语言中的 `%`。上面计算的是 $m = 10$ 的情况。

$$(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$$

证明：根据**带余除法**，任意整数 a 都可以表示为 $a = qm + r\ (m \neq 0)$，其中整数 q 为 a 除以 m 的商（quotient），整数 r 为 a 除以 m 的余数（remainder），即 $r = a \bmod m$。

设整数 $a = q_1 m + r_1$, $b = q_2 m + r_2$。

第一个恒等式：

$$\begin{aligned}
&(a+b) \bmod m \\
=&((q_1+q_2)m + r_1 + r_2) \bmod m \\
=&(r_1 + r_2) \bmod m \\
=&((a \bmod m) + (b \bmod m)) \bmod m
\end{aligned}$$

第二个恒等式：

$$\begin{aligned}
&(a \cdot b) \bmod m \\
=&(q_1 q_2 m^2 + (q_1 r_2 + q_2 r_1)m + r_1 r_2) \bmod m \\
=&(r_1 r_2) \bmod m \\
=&((a \bmod m) \cdot (b \bmod m)) \bmod m
\end{aligned}$$

根据这两个恒等式，我们可以在计算过程中（例如循环），对加法和乘法的结果取模，而不是在循环结束后再取模。

注：如果涉及到幂运算，指数是不能随意取模的。如果指数在 64 位整数的范围内，可以用**快速幂**计算，原理见【图解】一张图秒懂快速幂；如果指数超出 64 位整数的范围，见 欧拉降幂。

如果计算过程中有减法，可能会产生负数，处理不当也会导致 WA。如何正确处理这种情况呢？

同余

首先引入**同余**（congruence modulo）的概念。

两个整数 a 和 b，如果 $(a-b) \bmod m = 0$（也就是 $a-b$ 是 m 的倍数），则称 a 与 b 关于模 m 同余，记作

$$a \equiv b \pmod m$$

上式也称作**模 m 的同余式**，简称**同余式**。

例如 $42 \equiv 12 \pmod{10}$，$-17 \equiv 3 \pmod{10}$。

特别地，有

$$m \equiv 0 \pmod m$$

定理：在同余式两边乘以同一个整数，同余式仍然成立。

证明：设 $a \equiv b \pmod m$，设 k 是一个整数。根据同余的定义，$a-b$ 是 m 的倍数，所以 $k(a-b) = ka - kb$ 也是 m 的倍数，所以 $ka \equiv kb \pmod m$。

同余式的移项

同余式中的加减法可以移项。

例如

$$a+b \equiv c+d \pmod m$$

可以移项得到

$$a-c \equiv d-b \pmod m$$

证明： $a+b \equiv c+d \pmod m$ 意味着 $(a+b)-(c+d)$ 是 m 的倍数，即 $(a-c)-(d-b)$ 是 m 的倍数，所以 $a-c \equiv d-b \pmod m$。

推论： 在同余式两边加上（减去）同一个整数，同余式仍然成立。

证明： 设 $a \equiv b \pmod m$，左边加上 $0 = x - x$，得到 $a+(x-x) \equiv b \pmod m$，移项得 $a+x \equiv b+x \pmod m$。

特别地，在同余式

$$0 \equiv m \pmod m$$

的两边加上 $-x$，得

$$-x \equiv m-x \pmod m$$

例如在无符号 32 位整数中，-1 溢出得到 $2^{32}-1 = 4294967295$，用数学语言描述，就是

$$-1 \equiv 4294967295 \pmod{2^{32}}$$

负数和减法的取模

根据同余的定义，我们有 $-17 \equiv 3 \pmod{10}$。

怎么从 -17 得到 3？

我们可以 $-17+10+10=3$。也可以 $-17 \bmod 10 + 10 = -7+10 = 3$，这样只需加一次 m。

一般地，如果 $x<0$ 且 $0 \le y < m$，则 $x \equiv y \pmod m$ 相当于

$$x \bmod m + m = y$$

也就是用「模 m 加 m」，把 x「调整」为非负数。

为避免判断 $x<0$，更一般的写法是

$$(x \bmod m + m) \bmod m$$

这样无论 x 是正是负还是零，运算结果都会落在区间 $[0, m-1]$ 中。

对于减法来说，当 $a-b \ge 0$ 时，前文中的加法恒等式可以调整为

$$(a-b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

> 注 1：代码实现时，在计算中产生负数也可以，在最后用 $(x \bmod m + m) \bmod m$ 调整就行。
>
> 注 2：Python 用户可以忽略，只要 m 是正整数，取模运算的结果就一定是非负整数。

以上是关于加法、减法和乘法的模运算规则，那么除法呢？

除法的取模

如果要计算 $\frac{24}{8} \bmod 5$，可以像加法或乘法那样，写成 $\frac{24 \bmod 5}{8 \bmod 5} \bmod 5$ 吗？这不行，$\frac{24 \bmod 5}{8 \bmod 5} = \frac{4}{3}$ 甚至不是一个整数。

先说结论，如果 p 是一个质数，a 是 b 的倍数且 b 和 p 互质（b 不是 p 的倍数），那么有

$$\frac{a}{b} \bmod p = (a \cdot b^{p-2}) \bmod p$$

上式中 a 和 b 可以是很大的数，例如 $a = 100!,\ b = 50!50!$。

由于 10^9+7 是一个质数，所以上式可用于要求对 10^9+7 取模的题目。如果推导出了包含除法的式子，可以用上式转换成乘法，并用快速幂计算 $b^{p-2} \bmod p$。

下面是证明。

引理 1： 当 p 是质数且 $1 \le i \le p-1$ 时，有

$$\binom{p}{i} \equiv 0 \pmod p$$

其中

$$\binom{p}{i} = \frac{p!}{i!(p-i)!}$$

证明： 注意当 $1 \le i \le p-1$ 时，$\frac{p!}{i!(p-i)!}$ 的分母是不包含 p 的。由于分子包含 p 且 $\frac{p!}{i!(p-i)!}$ 是整数，所以 $\binom{p}{i}$ 可以被 p 整除，即 $\binom{p}{i} \equiv 0 \pmod p$。

注： 如果 p 不是质数，分母可能被 p 整除，上面的结论不一定成立。例如 $p=4,\ i=2$ 的情况，$\binom{p}{i} = 6$，不是 $p=4$ 的倍数。

引理 2： 对于任意整数 x 和 y 和任意质数 p，有

$$(x+y)^p \equiv x^p + y^p \pmod p$$

证明： 根据二项式定理，有

$$(x+y)^p = \binom{p}{0}x^p y^0 + \binom{p}{1}x^{p-1}y^1 + \binom{p}{2}x^{p-2}y^2 + \cdots + \binom{p}{p-1}x^1 y^{p-1} + \binom{p}{p}x^0 y^p$$

根据引理 1，除了第一项和最后一项以外，其余项都是 p 的倍数，于是

$$(x+y)^p \equiv x^p + y^p \pmod p$$

定理（费马小定理）：对于任意整数 b 和任意质数 p，有

$$b^p \equiv b \pmod p$$

证明： 当 $b=0$ 时，$0^p \equiv 0 \pmod p$ 成立。

假设 $b=k$ 时原命题成立，即

$$k^p \equiv k \pmod p$$

根据引理 2，我们有

$$(k+1)^p \equiv k^p + 1^p \pmod p$$

根据归纳假设，得

$$(k+1)^p \equiv k+1 \pmod p$$

即当 $b = k+1$ 时，原命题成立。

根据数学归纳法，原命题对于 $b \ge 0$ 成立。对于 $b < 0$ 的情况同理。

$b^p \equiv b \pmod p$ 变形得 $b(b^{p-1}-1) \equiv 0 \pmod p$，如果 b 不是 p 的倍数，那么 $b^{p-1}-1$ 必须是 p 的倍数（注意 p 是质数），即 $b^{p-1}-1 \equiv 0 \pmod p$，移项得

$$b^{p-1} \equiv 1 \pmod p$$

两边同时乘以 $\frac{a}{b}$（a 是 b 的倍数），得

$$a \cdot b^{p-2} \equiv \frac{a}{b} \pmod p$$

即

$$\frac{a}{b} \bmod p = (a \cdot b^{p-2}) \bmod p$$

> 注：除以 b 相当于乘以 b 的**逆元** $b^{-1} \bmod p$。在概率期望等题目中，会遇到 a 不是 b 的倍数的情况，这些题目通常会规定计算 $(a \cdot b^{-1}) \bmod p$，计算方法和上式一样。

总结

代码实现时，上面的加减乘除通常是这样写的：

```
MOD = 1_000_000_007

// 加
(a + b) % MOD

// 减，b 在 [0,MOD-1] 中，通过 +MOD 保证结果非负
(a - b + MOD) % MOD

// 把任意整数 a 取模到 [0,MOD-1] 中，无论 a 是正是负
(a % MOD + MOD) % MOD

// 乘（注意使用 64 位整数）
a * b % MOD

// 多个数相乘，要步步取模，防止溢出
a * b % MOD * c % MOD

// 除（MOD 是质数且 b 不是 MOD 的倍数）
a * qpow(b, MOD - 2, MOD) % MOD
```

其中 qpow 为**快速幂**，具体请看【图解】一张图秒懂快速幂。

> 注：Python 内置快速幂函数 `pow(x, y, m)` 用于计算 $x^y \bmod m$。特别地，除法也可以写成 `a * pow(b, -1, MOD) % MOD`。

总之，如果发现解答错误，可以检查下代码，看看是不是哪里漏掉取模了。

附：组合数的计算

关于组合数，我们需要预处理阶乘及其逆元，然后利用公式

$$C(n,m) = \frac{n!}{m!(n-m)!} = n! \cdot \frac{1}{m!} \cdot \frac{1}{(n-m)!}$$

计算。

对于阶乘 $n!$，可以用

$$n! = (n-1)! \cdot n$$

递推计算。

对于阶乘的倒数 $\frac{1}{n!}$，可以先计算 $N!$ 的逆元（其中 N 是 n 的最大值），然后用

$$\frac{1}{(n-1)!} = \frac{1}{n!} \cdot n$$

倒着递推计算。

模板代码如下：

```
Python3 | Java | C++ | Go

MOD = 1_000_000_007
MX = 100_001  # 根据题目数据范围修改

fac = [0] * MX  # fac[i] = i!
fac[0] = 1
for i in range(1, MX):
    fac[i] = fac[i - 1] * i % MOD

inv_f = [0] * MX  # inv_f[i] = i!^-1
inv_f[-1] = pow(fac[-1], -1, MOD)
for i in range(MX - 1, 0, -1):
    inv_f[i - 1] = inv_f[i] * i % MOD

# 从 n 个数中选 m 个数的方案数
def comb(n: int, m: int) -> int:
    return fac[n] * inv_f[m] * inv_f[n - m] % MOD if 0 <= m <= n else 0

class Solution:
    def solve(self, nums: List[int]) -> int:
        # 预处理的逻辑写在 class 外面，这样只会初始化一次
```